

Object Detection for P&ID Images using various Deep Learning Techniques

Mihir Gada

Synclovis Systems Pvt. Ltd.

Bangalore, India.

mihir.gada@synclovis.com

Abstract—Piping and Instrumentation Diagrams are detailed diagrams that show the piping and process equipment together with the instrumentation and control devices used in the process industry. These are updated during the plant life cycle to depict the latest changes and modifications done in the plant. These diagrams are detailed and are usually in the form of PDF, they are difficult to modify, and inferring details from these diagrams requires a deep understanding of plant process and engineering. To reduce the difficulty in finding and retrieving information from the diagrams, the proposal of automating the task of detection of the symbols and the texts embedded is explored in this paper. This paper compares various Deep Learning models on P&I Diagrams by using object detection for symbols using the Transfer Learning technique. Initially, the Connectionist Text Proposal Network algorithm is used for text detection in the images, then for the geometrically shaped objects, OpenCV library is used. The models trained by Transfer Learning are exported and the results are compared by performing symbol detection. This paper is an overall study of various deep learning methods implemented on P&I Diagrams.

Keywords—Piping and instrumentation diagrams; CTPN; SSD; Hough Transform; Faster R-CNN; R-FCN; OpenCV;

I. INTRODUCTION

Piping and instrumentation diagrams (P&ID) refers to a process flow chart during which equipment, piping, and instruments of a particular process are described in a structured form. All industrial process plants archive design drawings diagrammatic within the format as Portable Document Format (PDF) or some other vector graphics digitization process of conventional design drawings. Transferring or sharing these diagrams are either as imaged piping and instrumentation diagram (P&ID) drawings in common formats like portable network graphic (PNG), joint photographic experts group (JPG), and/or PDF [1]. Most industries maintain these complex P&IDs within the kind of hard-copies or scanned images and don't have an automatic mechanism for information extraction and analysis of P&IDs [2][3]. It would be of great value if the information present in P&ID sheets can be automatically extracted. However, Deep Learning has recently shown incredible lead in several key vision tasks like segmentation, classification, and generation of images [20].

This paper aims to scale back the matter in reading the diagrams and finding the symbols and hopefully improve the knowledge extraction from these P&ID diagrams. Initially, the text is extracted from P&ID sheets using the CTPN algorithm [5] then applying basic methods ranging from the OpenCV library [13] with Hough Transform function [12] to detect instruments to varied Deep Learning models like Faster Region-based Convolutional Neural Networks (Faster R-CNN) [9], Region-based Fully Convolutional Network (R-FCN) [10], and Single Shot Detection (SSD) [11] using the Transfer Learning technique and compare the models when applied on P&ID sheets.

II. BACKGROUND

A piping and instrumentation diagram (P&ID) could be a key drawing widely utilized inside the energy trade as well as fossil fuel and power plants. P&ID is supported by a process flow diagram (PFD), together they are an in-depth schematic illustration of the final flow of materials and their physical conditions [4].

Piping and instrumentation diagrams do not solely have diagrams inside the design of instruments and symbols however the additional text/ label/ code or tag is embedded alongside, this information is of importance and often referred to identify the physical device in the plant. At first, all the information is extracted within a text using the Connectionist Text Proposal Network (CTPN) [5]. It explores the detailed and contextual information of an image and is powerful to observe even ambiguous texts.

There are very few datasets and standards available for P&I diagrams since most are proprietary so we created our dataset from a sample P&ID sheet. The creation of a model from scratch may give rise to the problem of overfitting due to the small reference set of images in the dataset. Using the Transfer Learning technique we prevent the overfitting problem by storing the knowledge gained while solving one problem-related related to the domain and applying them for being employed in current tasks [6]. The TensorFlow Object Detection API contains various pre-trained models that have been trained on MS-COCO [8], a dataset for image recognition, segmentation, and captioning, consisting of more than 300,000 images overall and 80 object classes. Hence, object detection models like SSD, Faster R-CNN, and R-FCN

have been used. Before that, the OpenCV[13] library having Hough Transformation Function[12] was used on the sheets to detect all the instrument symbols with geometrical shapes, especially circles.

Talking about the Object Detection Models, they can be divided into two categories such as One-shot or Two-shot can be referred to as Regression-based and Regional Based as explained in the paper [7]. Few of One Shot Detection Models are ‘You Only Look Once’ (YOLO) and ‘Single Shot Detection’ (SSD) whereas Two Shot Detection Models are ‘Region-based Convolutional Neural Networks’ (RCNN), Fast-RCNN, Faster R-CNN, Region-based Fully Convolutional Networks (R-FCN), and Spatial Pyramid Pooling network (SPP-net). The details of Faster R-CNN, R-FCN, and SSD are listed below as per the reference paper [7]:

a) Faster R-CNN employs the region proposal network (RPN) to resolve the problems of giant computation and poor time caused by the selective search technique in R-CNN and Fast R-CNN.

b) R-FCN [10], is a full convolution neural network based on regions, having solved the problem that Region of Interest (RoI) can’t share the computation. It also adopts Region Proposal Network to generate candidate ROIs. As for the performance, the accuracy of R-FCN is similar to that of Faster R-CNN, but the test speed of R-FCN is 2.5 times that of the Faster R-CNN.

c) SSD [11] is the single-shot detector such that the design of SSD has integrated YOLO’s regression idea [21] and Faster R-CNN’s anchor mechanism. With the regression plan of YOLO, SSD simplifies the computation quality of the neural network to substantiate the period. It eliminates proposal generation associated future element or feature resampling stages and encapsulates all computation in a passing single network [22].

III. RELATED WORK

There have been few works on object detection and feature recognition in Piping and instrumentation diagrams but the study has not been comparative based on various techniques to find the best solutions for different types of P&ID sheets.

As mentioned in [14] methods detecting symbols and texts in P&ID images using geometrical and deep learning-based approaches, the text detection accuracy was about 89.1%. A Faster R-CNN model was chosen for a single class i.e. pump and 9 test images along with 10 train images were used. There is a lack of the post-implementation of the model after it was completely trained and the use of any other methods or models instead of the Faster-RCNN are not mentioned.

The paper[2] proposes various methods for feature extraction from P&ID Sheets with a text detection accuracy of 90.1%. Pipeline detection is finished with an accuracy of 65.2% due to random noise from line-markings and overlaid diagrams. For symbol detection, the VGG-19 [18] based FCN [19] was used with multiple classes giving an F1 score of more than

0.86 and precision 100% for 5 out of 10 the symbols. The results were obtained due to the availability of the dataset along with variations but with P&ID being industrial diagrams there is no public domain dataset. Thus to handle this issue of less dataset, Transfer Learning comes to the rescue.

IV. METHODOLOGY

In this section, we tend to discuss the projected methodology for extracting data from P&ID sheets thoroughly. Starting with text detection using CTPN, using OpenCV library having Hough Transform function, and finally using the Transfer Learning Technique on three types of object detection models like SSD, Faster R-CNN, R-FCN.

A. Text Detection

A P&ID typically contains text data indicating ids (Tag), numbers, and connections of equipment, pipelines, control, electrical connections, and instruments. Since the diagrams are large and complicated to understand, directly applying OCR[15] will not give the required results, hence once texts in a P&ID are detected using CTPN, their information can be extracted through OCR technologies and then removed from the P&ID to reduce unnecessary noise for a deep learning-based symbol detection training. The Connectionist Text Proposal Network(CTPN)[4] model is ready by the VGG net pre-trained model with ICDAR 2013 MLT dataset to find texts in an exceedingly sample P&ID. After the model is trained for 50,000 iterations, it is exported and applied to the images. Once the coordinates of the texts in P&ID are localized correctly, then OCR technology such as Tesseract[15] is applied to recognize the localized texts.

B. Object Detection

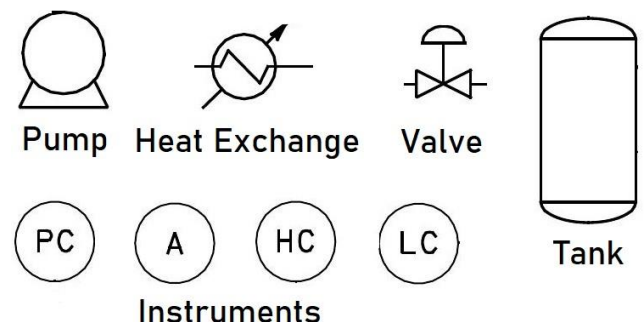


Figure 1. Few of the important objects in P&ID.

a) Instrumentation symbol Detection

Instrumentation symbols are simple geometric shapes, as shown in Figure 1. with texts embedded inside them describing the type of the instrument. Texts initially can be detected using CTPN as explained in the previous section (4.1). To detect instruments we use Python’s OpenCV[13] library having Hough transform function[12]. The parameters that were adjusted are:

1. *Image*: It is an 8bit, single-channel image, and here we have all grayscale images if these aren't then it is preferred to convert images to grayscale.
2. *Method*: It defines the method to detect circles in images. Currently, the sole enforced methodology is cv2.HOUGH_GRADIENT, that corresponds to the paper [17]
3. *dp*: This parameter is the inverse ratio of the accumulator resolution to the image resolution [17]. The dp is inversely proportional to the accumulator array. Larger the dp, the smaller the accumulator array
4. *minDist*: Minimum distance between the circles is controlled by this parameter.
5. *param1*: Gradient value used to handle edge detection in the [17]method.
6. *param2*: Accumulator threshold value for the cv2.HOUGH_GRADIENT method. More circles will be detected including the false if the threshold is smaller. More circles may be returned if the threshold value is larger.
7. *minRadius*: Minimum size of the radius (in pixels).
8. *maxRadius*: Maximum size of the radius (in pixels)

As the P&ID sheets are of large size the parameters values were taken into consideration are dp=1, minDist=20, param1=50, param2=30, minRadius=30, and maxRadius=50.

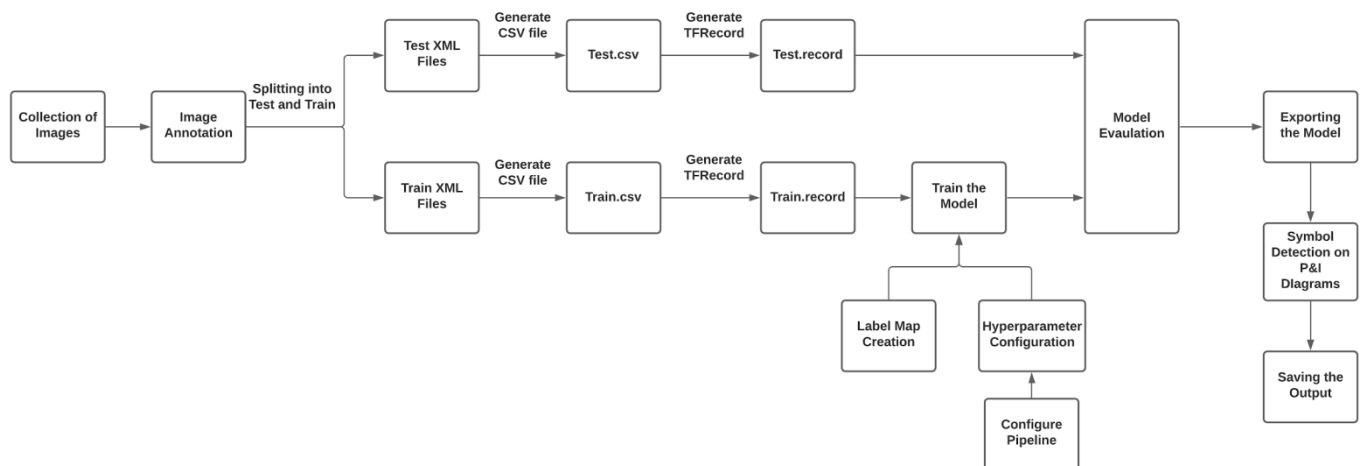


Figure 2. Flow Diagram for Symbol Detection

b) Symbols Detection

Figure 2 comprises the complete flow of the process of how symbols are detected starting from the collection of images to saving the output into a file.

1. Collection of images

The collection of P&IDs is in PDF format so they were first converted to JPG format. P&ID sheets are large 6800*4400pixels so they were first cropped by removing rulers and tables on the side and then divided into images of 1000*1000pixels. These contained multiple classes of symbols in a single image.

2. Annotating the images

The images were then labeled using LabelImg [23], an open-source python-based tool to label images graphically. The tool can annotate in both eXtensible Markup Language (XML) and PASCAL Visual Object Classes (VOC) format, here we use the XML format.

3. Splitting the images

After annotation is complete, the labels need to be split into train and test sets of labels with a ratio of 7:3 respectively. The

train-test split procedure is used to estimate the performance of machine learning algorithms to make predictions effectively on test data.

4. Preprocessing Images and Labels

This section processes the images and labels to the TFRecords file format. [24] which is Tensorflow's standard file format storing a sequence of binary records. Data is in one block of memory, which makes it easier to distribute and make the data better aligned for efficient reading. To convert to TFRecord file format, firstly the split data which is in XML format along with the path to images is converted to CSV format as processing becomes easier. Then it is processed to the TFRecord file format.

5. Training Details

The TensorFlow Object Detection API contains various pre-trained models that have been trained on MS-COCO [8], we chose three pre-trained models like Single-Shot Detection (SSD) network with Mobilenet as feature extractor, Region-based Fully Convolutional Networks with ResNets as feature extractor, and Faster regions with convolutional neural networks with Inception as feature extractor intended to

perform object detection. Input images of size 1000*1000px converted to 300*300px were fed to the network and trained at a learning rate of 0.003 and batch size 12. The models were trained for 30,000 steps.

6. Evaluation

After models were trained they were exported for inference on various P&ID images. The symbols were detected and the data extracted in the form of coordinates along with the output image were stored.

V. RESULTS

This section comprises all the results obtained from the training and implementation. Due to the confidentiality of the data in the images, the images are cropped and shown here.

A. Text Detection

After implementing CTPN on the images we can see in Figure 3. all the texts are detected accurately and their positions are being saved in a text file for future use.

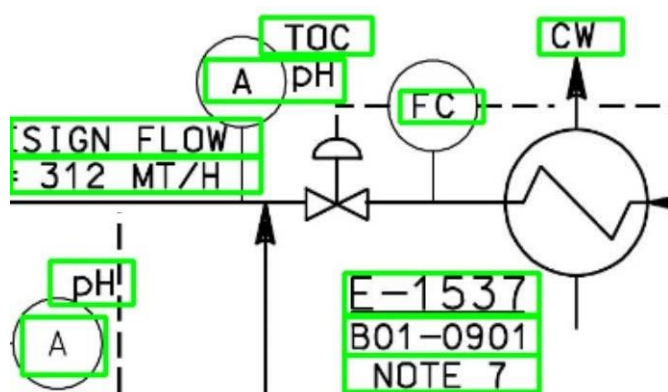


Figure 3 Text Detection using CTPN

B. Object Detection

a) Instruments

Using the OpenCV Library having Hough Transform function we can detect all the instruments in the diagrams as shown in Figure 4. As we can see that circles as part of the text were not detected because of parameter selection. This paper [14] guides to overcome the situation where texts were also detected as symbols.

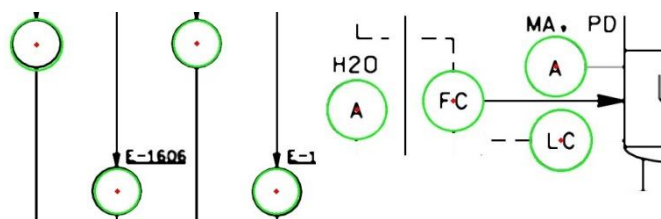


Figure 4. Instrument Detection

b) Symbols

After Completing the Process of training all the three models, they were exported and then applied on the P&ID sheets. Given the training results in Table I, Faster R-CNN proves the best of SSD and R-FCN. But when a large no. of images are to be processed, then SSD and R-FCN prove faster than Faster R-CNN and gives the most accurate results. R-FCN being a Two-Shot Detector proves better in accuracy and matches similar speed with SSD. But SSD outperforms all if the size of the symbols is large.

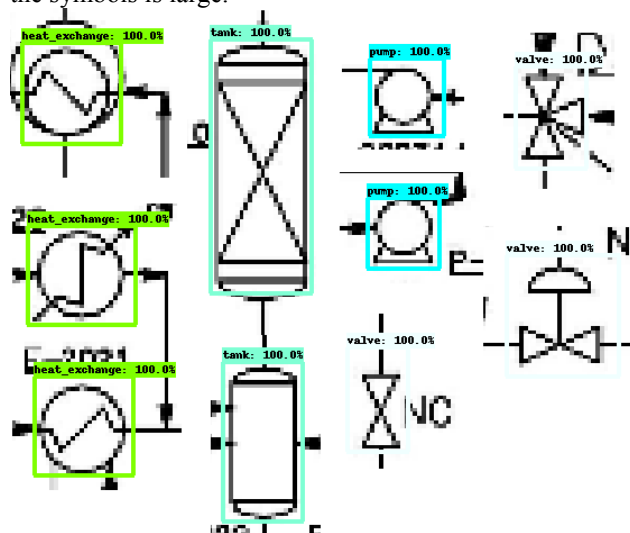


Figure 5. Symbol Detection

TABLE I. Model Comparison Chart

Models	Metric			
	Precision	Recall	F1 Score	Accuracy
SSD	82.51%	49.95%	62.22%	98%
Faster R-CNN	86.63%	53.45%	66.11%	99%
R-FCN	84.76%	52.60%	64.91%	98%

VI. CONCLUSION

Hence, in this paper, information is extracted from the piping and instrumentation diagrams using various Deep Learning Techniques, and model performance comparison is done for object detection on the images. Initially, text detection is performed in P&I Diagrams using deep learning networks like the CTPN algorithm. Instruments in the P&ID images are detected using the OpenCV library. Dataset is created and trained on models like SSD, Faster R-CNN, and R-FCN using Transfer Learning techniques. The exported model is then implemented on various P&ID images and results conclude that Faster R-CNN is more accurate but slow whereas SSD and R-FCN are faster and are nearby in terms of accuracy with Faster R-CNN. But an SSD can outperform if the object size is large. So based on the input diagrams on which objects are to be detected, we can use appropriate detection models. All the

data is then saved in the form of a CSV file for the complete database creation and analysis.

ACKNOWLEDGMENTS

This paper and the research behind it would not have been possible without the exceptional support of Mr. Amit Pandey, CEO, Synclovis Systems Pvt. Ltd. I would like to extend my gratitude towards Mr. Nitin Kumar, who is working as a Consultant for Oil and Gas (AI/ML/Digital Twin Solutions Consultant) for coming up with the base idea for this research, guiding and mentoring in the drafting of the paper. Also, I would like to thank Mr. D Sudhakar Reddy, Software Engineer, Synclovis Systems Pvt. Ltd. for guiding me throughout my research.

REFERENCES

- [1] Kang S-O, Lee E-B, Baek H-K. A Digitization and Conversion Tool for Imaged Drawings to Intelligent Piping and Instrumentation Diagrams (P&ID). *Energies*. 2019; 12(13):2593.
- [2] Rahul, Rohit & Paliwal, Shubham & Sharma, Monika & Vig, Lovekesh. (2019). Automatic Information Extraction from Piping and Instrumentation Diagrams.
- [3] E. Arroyo, A. Fay, M. Chioua and M. Hoernicke, "Integrating plant and process information as a basis for automated plant diagnosis tasks," *Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA)*, Barcelona, 2014, pp. 1-8, doi: 10.1109/ETFA.2014.7005098.
- [4] Yu, & Cha, & Lee, & Kim, & Mun, Duhwan. (2019). Features Recognition from Piping and Instrumentation Diagrams in Image Format Using a Deep Learning Network. *Energies*. 12. 4425. 10.3390/en12234425.
- [5] Tian, Zhi & Huang, Weilin & Tong, He & He, Pan & Qiao, Yu. (2016). Detecting Text in Natural Image with Connectionist Text Proposal Network. 9912. 56-72. 10.1007/978-3-319-46484-8_4.
- [6] L. Shao, F. Zhu and X. Li, "Transfer Learning for Visual Categorization: A Survey," in *IEEE Transactions on Neural Networks and Learning Systems*, vol. 26, no. 5, pp. 1019-1034, May 2015, doi: 10.1109/TNNLS.2014.2330900.
- [7] C. Tang, Y. Feng, X. Yang, C. Zheng and Y. Zhou, "The Object Detection Based on Deep Learning," 2017 4th International Conference on Information Science and Control Engineering (ICISCE), Changsha, 2017, pp. 723-728, doi: 10.1109/ICISCE.2017.156.
- [8] Lin, Tsung-Yi & Maire, Michael & Belongie, Serge & Hays, James & Perona, Pietro & Ramanan, Deva & Dollár, Piotr & Zitnick, C.. (2014). Microsoft COCO: Common Objects in Context.
- [9] S. Ren, K. He, R. Girshick and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137-1149, 1 June 2017, doi: 10.1109/TPAMI.2016.2577031.
- [10] Y. Li, K. He, J. Sun, "R-FCN: Object detection via region-based fully convolutional networks," *Advances in Neural Information Processing Systems*, 2016, pp. 379-387.
- [11] Liu W. et al. (2016) SSD: Single Shot MultiBox Detector. In: Leibe B., Matas J., Sebe N., Welling M. (eds) *Computer Vision – ECCV 2016*. ECCV 2016. Lecture Notes in Computer Science, vol 9905. Springer, Cham. https://doi.org/10.1007/978-3-319-46448-0_2
- [12] Shehata, Allam & Mohammad, Sherien & Abdallah, Mohamed & Ragab, Mohammad. (2015). A Survey on Hough Transform, Theory, Techniques and Applications.
- [13] I. Culjak, D. Abram, T. Pribanic, H. Dzapo and M. Cifrek, "A brief introduction to OpenCV," 2012 Proceedings of the 35th International Convention MIPRO, Opatija, 2012, pp. 1725-1730.
- [14] Cha, Jae-Min & Lee, Taekyong & Salim, Shelly & Ryu, Bohyun. (2019). Object Detection for P&ID Images.
- [15] Smith, R.. (2007). An Overview of the Tesseract OCR Engine. Ninth International Conference on Document Analysis and Recognition (ICDAR 2007). 2. 629 - 633. 10.1109/ICDAR.2007.4376991.
- [16] Rosebrock, Adrian. "Detecting Circles in Images Using OpenCV and Hough Circles - PyImageSearch." *PyImageSearch*, 18 Apr. 2020, pyimagesearch.com/2014/07/21/detecting-circles-images-using-opencv-hough-circles/.
- [17] H. K. Yuen, J. Princen, J. Illingworth, & J. Kittler (1989). A comparative study of Hough transform methods for circle finding. In *Proc. 5th Alvey Vision Conf.*, Reading (31 Aug (pp. 169–174).
- [18] Simonyan, Karen & Zisserman, Andrew. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv* 1409.1556.
- [19] Shelhamer, Evan & Long, Jonathon & Darrell, Trevor. (2016). Fully Convolutional Networks for Semantic Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 39. 1-1. 10.1109/TPAMI.2016.2572683.
- [20] Liu, L., Ouyang, W., Wang, X. et al. Deep Learning for Generic Object Detection: A Survey. *Int J Comput Vis* 128, 261–318 (2020). <https://doi.org/10.1007/s11263-019-01247-4>
- [21] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.
- [22] Zhao, Xiaotong & Li, Wei & Zhang, Yifan & Feng, Zhiyong. (2018). Residual Super-Resolution Single Shot Network for Low-Resolution Object Detection. *IEEE Access*. PP. 1-1. 10.1109/ACCESS.2018.2867586.
- [23] Lin, TzuTa. LabelImg: LabelImg Is a Graphical Image Annotation Tool and Label Object Bounding Boxes in Images. *PyPI*, <https://github.com/tzutalin/labelimg>.
- [24] "TFRecord and Tf.Example | TensorFlow Core." TensorFlow, https://www.tensorflow.org/tutorials/load_data/tfrecord.